

Relatório - Aula 007

Expressões, Atribuição e Controle

Objetivo

O objetivo primário é compreender como as linguagens avaliam expressões e gerenciam o fluxo de controle para evitar surpresas no código. O slide apresenta casos reais em que falhas de controle causaram problemas graves, como a verificação pulada do "goto fail" da Apple, os travamentos dos aparelhos Zune da Microsoft em anos bissextos e vulnerabilidades de senhas no PHP ocasionadas por comparações frouxas com `==`

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela endentação ou pela precedência
- evitar conversões implícitas perigosas (`==` do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle:

2014 · "goto fail" (Apple)

Uma linha goto fail; duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um laço while nunca terminava, e todos os aparelhos travaram.

PHP · comparação com `==`

"0e4..." `==` "0e8..." é verdadeiro: logins que comparavam hashes com `==` aceitavam senhas erradas.

O que cada linha imprime?

python:

```
print(-2 ** 2, 2 ** 3 ** 2)
```

A saída é -4 e 512

Java :

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

A saída é 3345.

JavaScript

```
console.log("5" - 2, "5" + 2);
```

A saída é 3 e "52".

C

```
printf("%d\n", 3 > 2 > 1);
```

A saída é 0

Python

```
x = 0
```

```
print(x or "padrão", 3 > 2 > 1)
```

A saída é "padrão" e True

JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

A saída é true e false.

Expressões aritméticas: questões de projeto

. Quais são as regras de precedência?

R: As regras de precedência definem a ordem prioritária em que operadores diferentes são calculados dentro de uma mesma expressão. Linguagens com notação prefixa como o Lisp dispensam regras de precedência convencionais.

. Quais são as regras de associatividade?

R: As regras de associatividade definem o sentido de execução (da esquerda para a direita, ou o oposto) em casos de operadores que detêm o mesmo nível de precedência. A linguagem APL, por exemplo, avalia sentenças exclusivamente da direita para a esquerda.

. Em que ordem os operandos são avaliados?

R: Este questionamento define a ordem de processamento cronológico com a qual o compilador carrega as variáveis na memória no ato da avaliação.

. Há restrições a efeitos colaterais?

R: Analisa a política da linguagem de permitir ou bloquear interferências em estados externos à função enquanto uma conta está sendo executada.

. O usuário pode sobrecarregar operadores?

R: Verifica se o programador tem autorização do sistema para reescrever e atribuir novos significados a símbolos nativos em seus tipos próprios.

. Que mistura de tipos é permitida?

R: Consiste nas políticas do compilador que aceitam (ou negam) a manipulação conjunta e implícita de variáveis formatadas diferentemente em um mesmo bloco aritmético.

Precedência e associatividade na prática

Diferentes ferramentas tratam associatividade diferentemente; Python resolve a exponenciação `**` orientada à direita, enquanto o Excel atribui peso superior ao menos unário na exponenciação `^`. Em Javascript, misturar negativos e exponenciações sem parênteses resulta num erro de sintaxe explícito, demonstrando que o uso de parênteses explicita intenções e auxilia todos os compiladores.

Ordem dos operandos e efeitos colaterais

A linguagem Java possui garantias nativas absolutas que forçam a execução partindo da esquerda em direção à direita. O padrão em Go e em C não define esse controle restrito, potencializando anomalias perigosas de processamento na presença de lógicas contendo efeitos colaterais. Em uma codificação com pureza e transparência referencial a ordem perfeitamente imutável para a obtenção do resultado matemático não faria diferença.

Operadores sobrecarregados

Usar o mesmo caractere para múltiplas intenções pode gerar perigos, como o `&` sendo "E bit a bit" ou "endereço de" em C++. Python permite a configuração de operadores de classe pelo programador com atalhos como o `__add__` interno. Java, por sua vez, bloqueia categoricamente sobrecargas feitas livremente pelo usuário.

Conversões de tipos: alargamento, estreitamento e coerção

O comportamento da tipagem mista varia desde tolerância implícita silenciosa em C (inclusive para estreitamentos arriscados) até aceitação limitada de alargamentos de base de dados em Java. Arquiteturas modernas seguras como F#, Go e Rust repudiam e falham na mescla de coerção automática, exigindo declaração manual via *cast* sob pena de erro no compilador.

Coerção implícita: JavaScript e PHP

O JavaScript traz coerções ilógicas complexas onde `null + 1` vira 1, gerando vulnerabilidades lógicas complexas. Em PHP o mecanismo batizado como "type juggling" é responsável por autenticações forjadas acidentalmente ao comparar chaves criptográficas que iniciam com numerais grandes pela marcação de base e erro no igual duplo simples. Sugere-se o comparador absoluto (`===`) nestas linguagens.

Expressões relacionais e booleanas

Em linguagens antigas baseadas em C a averiguação booleana é um mapeamento inteiro, e isso resulta em falhas em condições de escopo aglutinado onde `3 > 2 > 1` resolve falsamente como `0` numérico. Linguagens de nível abstrato alto (Python) conseguem alinhar comparações conjuntas, já Java e Rust proíbem literais indevidos limitando condicionais a valores inerentemente booleanos lógicos.

Avaliação em curto-circuito

Técnica de barramento rápido presente com a checagem com `&&` do Java para contornar perigos em partes da sentença secundária na verificação condicional. Ferramentas atuais expandem essa mecânica pelo Javascript sob a sintaxe `?.` e `??`. O perigo real ocorre se a parte desprezada pelo salto carregar operadores mutáveis e cruciais ao estado rotineiro do sistema como `b++`.

Sentenças de atribuição

Enquanto implementações em C e JavaScript tratam a atribuição em si como uma sentença geradora de retorno validada tacitamente no `if (x = y)`, essa liberdade foi banida das versões de Python originais e afins por propiciar armadilhas. Posteriormente em Python a condicional foi permitida sob uso único do operador restrito condicionado da morsa `:=`. Modelos focados na programação funcional tratam imutabilidades nas atribuições representadas nominalmente pelo marcador `let` e `val`.

Atribuição de modo misto

Em C, ocorrem coerções de tipagem entre variáveis para qualquer direção sem emitir alertas expressivos para o programador. Em Java, o sistema efetua alargamentos em silêncio de maneira automática, e uma armadilha trágica que ocorre é a quebra fracional oculta quando operações matemáticas inteiras precedem a mescla tipada de divisão.

Estruturas de controle: quantas são necessárias?

Fica evidenciado pelo fundamento arquitetural de Böhm e Jacopini de 1966 que a sequência, laços de repetição contidos e checagem de blocos são suficientes para resolver quaisquer demandas. Todos os atalhos a mais nas linguagens existem majoritariamente por motivo de otimização de trabalho humano buscando alavancar a legibilidade geral das estruturas visuais dispostas do compilador.

Seleção de dois caminhos: o "else pendente"

Configurou-se uma imprecisão clássica em C e no Java onde o compilador atribui o encerramento do `else` acompanhando implicitamente o bloco de checagem condicional `if` que ficou aberto no caminho precedente mais perto, enganando pelo desalinhamento visual estético não mapeado pela indentação livre. Perl, Go, Rust impõem soluções através das chaves rigorosamente delimitadas resolvendo a estética e semântica de ponta a ponta sem ambiguidade lógica na programação do recuo.

Seleção múltipla: do switch de C ao match

A verificação segmentada múltipla do C no condicional `switch` tem como comportamento nocivo natural prosseguir executando o interior caso a quebra do `break` omita o controle de escape da execução da condição (fallthrough). As saídas e correções incluem a mecânica de exaustão do tipo declarativo restrito sem queda de Rust através do `match`.

Laços controlados por contador

Iterações controladas divergem num modelo antigo maleável em C, permitindo expressões avulsas modificando a execução da checagem na trava condicional de escopo livre no controle geral do passo das variáveis limitadoras de avanço nas chaves. Diferente do iterável blindado por `for in` nas engrenagens atreladas internamente sem alteração possível exposta para segurança dos laços delimitadores no Python atual.

Laços lógicos, saídas e iteradores

JavaScript demonstra a grande diferença num desvio crítico separando o percurso entre `for...in` de chaves string numéricas lógicas de matrizes contra os retornos do `for...of` que trazem valores limpos literais contidos da listagem global. A atenção na limitação iteradora contorna as anomalias estagnadas mortas causadoras da quebra grave no equipamento Microsoft Zune em 2008 onde a contagem parava sem saída atestada no limite dos laços eternos no laço `do-while` contido nas chaves do laço de teste final oculto de execução única implícita ao condicional bloqueado.

Desvio incondicional: o goto

Descreve-se amplamente os estragos causados pelo modelo destrutivo atrelado à perda lógica do processamento abordados por Dijkstra que condenou essa quebra incondicional historicamente comprovada em desastres catastróficos no código Apple ("goto fail"). Contornos mais brandos em versões atuais gerenciaram esses recursos com as chamadas finalizadas e saídas explícitas limitadoras nomeadas que assumem papel brando de encerramentos limitados padronizados por `continue`, `return` e `break` restritivos contidos nos fluxos.

Comandos protegidos de Dijkstra

Ilustram uma condicional simultânea não amarrada exposta inicialmente pela ideia de checagem simultânea sem determinismo que avaliam todas as condições simultaneamente causando sorteios caso mais de uma guarda condicional seja declarada ou um erro final geral no laço caso seja completamente vazio. Esse modelo embasa os canais de simultaneidade nativamente atrelados na estrutura select processada concorrentemente baseado na checagem e bloqueio expostas em Go moderno nos processos do sistema de execução global de comandos limitadores protegidos declarativos internos nas rotinas limitadoras simultâneas de checagem.